

# Sanipatitas — HC Veterinaria

Arquitectura Orientada al Servicio

---

Alcántara Paico, Luis Sebastián

Contreras Delgado, Jorge Renzo Paolo

Huamán Domador, Luis Alberto

Ramos Yampufe, Martin Alexander

Ruiz Cardoza, Richard

2026-07-19

Universidad Tecnológica del Perú

## 1. Implementación Backend

---

## 1.1 Servicios del sistema

| Servicio               | Interfaz REST                                         | Implementación                                   |
|------------------------|-------------------------------------------------------|--------------------------------------------------|
| Auth Service           | POST /sign-in/email + GET /jwks<br>+ admin CRUD       | Bun + Elysia + Better Auth                       |
| Patient Service        | CRUD /patient + /client + /<br>species + /breed       | Java 25 + Quarkus 3.37.0 +<br>Hibernate Reactive |
| Appointment<br>Service | CRUD /appointment + GET /<br>appointment/events (SSE) | Java 25 + Quarkus 3.37.0 +<br>Hibernate Reactive |

Los tres servicios comparten librerías comunes, lo que garantiza consistencia en modelos de dominio, validaciones y middleware transversal.

# 1.2 API REST y base de datos

Cada servicio expone una API REST documentada con OpenAPI 3.0. Las especificaciones se generan automáticamente: SmallRye OpenAPI en Quarkus y Better Auth en Elysia.

| Servicio | Endpoint                     | Roles                       |
|----------|------------------------------|-----------------------------|
| Auth     | POST /api/auth/sign-in/email | público                     |
| Auth     | POST /api/auth/sign-up/email | público                     |
| Auth     | GET /api/auth/jwks           | público                     |
| Auth     | POST /api/auth/token         | público                     |
| Auth     | GET /api/auth/session        | autenticado                 |
| Auth     | */api/auth/admin/*           | admin                       |
| Patient  | GET /api/patient             | admin, veterinarian, worker |
| Patient  | POST /api/patient            | admin, worker               |
| Patient  | PUT /api/patient/{id}        | admin, worker               |
| Patient  | DELETE /api/patient/{id}     | admin                       |

# 1.2 API REST y base de datos

| Servicio    | Endpoint                     | Roles                       |
|-------------|------------------------------|-----------------------------|
| Appointment | GET /api/appointment         | admin, veterinarian, worker |
| Appointment | POST /api/appointment        | admin, veterinarian, worker |
| Appointment | PUT /api/appointment/{id}    | admin, veterinarian, worker |
| Appointment | DELETE /api/appointment/{id} | admin                       |
| Appointment | GET /api/appointment/events  | admin, veterinarian, worker |

PostgreSQL 17 con tsvector para texto completo en español y claves UUID v7.

## 1.3 Comunicación entre servicios

HAProxy expone un único punto de entrada y enruta según el prefijo de la ruta:

```
/api/auth          -> auth:3000  
/api/patient       -> patient:8080  
/api/appointment  -> appointment:8081  
/*                 -> frontend:1420
```

La comunicación inter-servicios se realiza mediante HTTP con propagación de token JWT. Cada servicio valida el token de forma autónoma consultando el endpoint JWKS del Auth Service, eliminando la dependencia de un punto central de validación en cada petición.

## 2. Integración Frontend

---

# 2.1 Stack del cliente

El frontend está construido con **Astro 6** (SSR con adaptador Node.js) y **React 19**. Se distribuye como PWA y como aplicación de escritorio mediante **Tauri** (Rust).

| Capa           | Tecnología                                         |
|----------------|----------------------------------------------------|
| Routing        | TanStack Router con code-splitting por archivo     |
| Estilos        | Tailwind CSS v4 + @base-ui/react + @sanipatitas/ui |
| Estado         | TanStack React Query + nanostores                  |
| Formularios    | React Hook Form + Zod                              |
| Tablas         | TanStack React Table                               |
| Cliente API    | @hey-api/openapi-ts generado desde OpenAPI         |
| Iconos         | React Icons (Tabler), SVG vía vite-plugin-svgr     |
| Notificaciones | Sonner (toasts)                                    |
| Temas          | next-themes (claro/oscuro)                         |
| PWA            | @vite-pwa/astro con Workbox                        |

## 2.2 Cliente API generado automáticamente

`@hey-api/openapi-ts` genera automáticamente tipos TypeScript, funciones HTTP y hooks de TanStack React Query a partir de los esquemas OpenAPI de cada servicio. Cada servicio expone su especificación en un endpoint dedicado:

- Auth: GET `/api/auth/openapi.json`
- Patient: GET `/api/patient/openapi.json`
- Appointment: GET `/api/appointment/openapi.json`

Un script de build (`generate-api-client`) consulta los tres endpoints y produce:

- `sdk.gen.ts` — funciones fetch tipadas
- `types.gen.ts` — interfaces TypeScript
- `zod.gen.ts` — esquemas Zod para validación
- `@tanstack/react-query.gen.ts` — hooks `useQuery` y `useMutation`

Cada petición incluye el JWT en `Authorization: Bearer {token}`, almacenado en cookie `httpOnly` (`sanipatitas.session_token`). Si el backend responde 401, se redirige al login.

## 2.3 Flujos de usuario principales

**Identificación por QR.** Escaneo del código sanipatitas://patient/{uuid} → extracción del UUID → GET /api/patient/{uuid} → ficha del paciente en panel lateral.

**Gestión de citas.** Selección de paciente y cliente → verificación de disponibilidad → registro de cita. Appointment Service publica evento SSE que actualiza la vista de todos los usuarios en tiempo real.

**Consulta de historial.** El veterinario accede a la ficha y consulta el historial. Las tablas de Hibernate Envers (patient\_aud, appointment\_aud) permiten reconstruir el historial completo de cambios.

### 3. Administración de Procesos y Auditoría

---

## 3.1 Servicio transversal de auditoría

Registro de transacciones críticas (POST, PUT, PATCH, DELETE) sobre pacientes, clientes, especies, razas, citas y autenticación. Cada evento registra:

- type: "audit" para distinguir de otros logs
- who: usuario del claim sub del JWT, y role del usuario
- when: timestamp ISO-8601 con milisegundos
- service: servicio de origen (patient, appointment, auth)
- action: tipo de operación (CREATE, UPDATE, DELETE)
- resourceType y resourceId: recurso afectado
- method, path, status y durationMs: metadatos HTTP
- correlationId: trazabilidad entre servicios

Implementación en tres capas:

**Logs en tiempo real.** AuditLogFilter en Quarkus y auditMiddleware en Elysia interceptan cada petición y emiten logs JSON con el mismo esquema de campos.

## 3.1 Servicio transversal de auditoría

**Hibernate Envers.** quarkus-hibernate-envers sobre una unidad JDBC dedicada. Cada entidad @Audited genera tablas \_AUD con snapshots históricos. SanipatitasRevisionEntity registra username y role por revisión. Tablas: revinfo, patient\_aud, client\_aud, appointment\_aud, breed\_aud, species\_aud, user\_aud.

## 3.2 Centralización en Loki y Grafana

Los servicios envían logs directamente a **Loki** mediante HTTP, sin plugins de Docker ni sidecars:

| Servicio              | Mecanismo                |
|-----------------------|--------------------------|
| Auth (Elysia/Bun)     | loki-transport.ts (Pino) |
| Patient (Quarkus)     | LokiLogHandler (JUL)     |
| Appointment (Quarkus) | LokiLogHandler (JUL)     |

LokiLogHandler se registra automáticamente mediante LokiLogSetup (@Observes @Initialized). Bufferea lotes de 100 registros y los envía asíncronamente. Variables: `LOKI_ENABLED=true`, `LOKI_URL`, `SERVICE_NAME`.

Los dashboards de Grafana se aprovisionan automáticamente. LogQL:

```
{service=~"patient|appointment|auth"} | json | type="audit"
```

## 4. Modelado de procesos con BPMN

---

## 4.1 Procesos modelados con Camunda 8

Se utilizó **Camunda 8** con **BPMN 2.0** para modelar los flujos de negocio del sistema. Se modelaron cinco procesos:

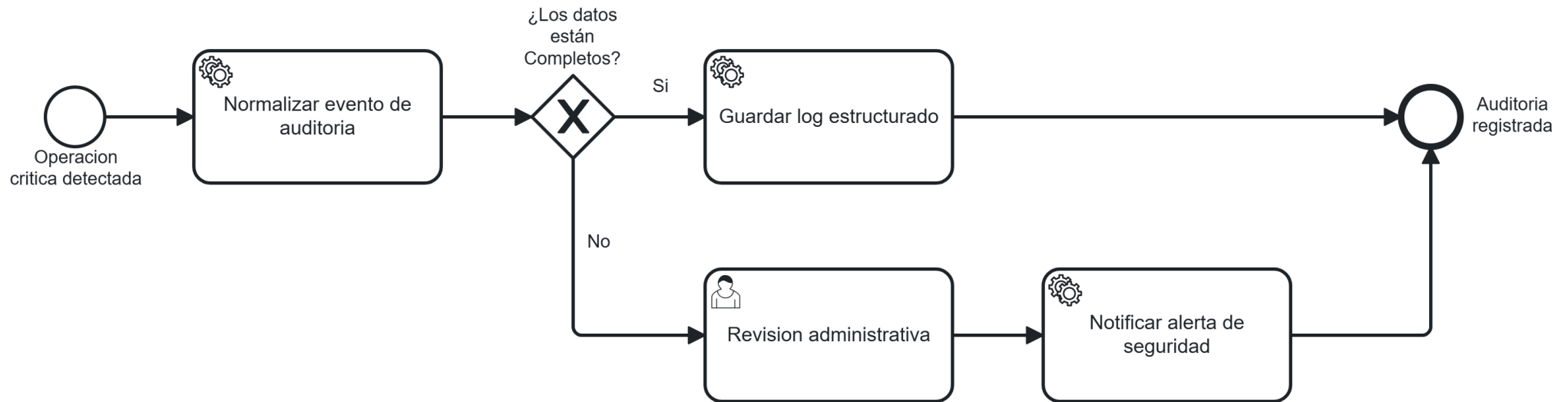


Figure 1: Auditoría de transacción crítica

## 4.1 Procesos modelados con Camunda 8

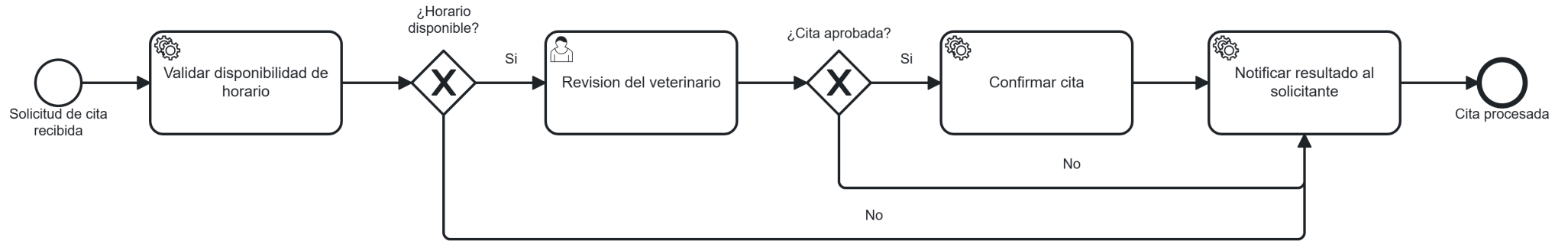


Figure 2: Aprobación de cita veterinaria

## 4.1 Procesos modelados con Camunda 8

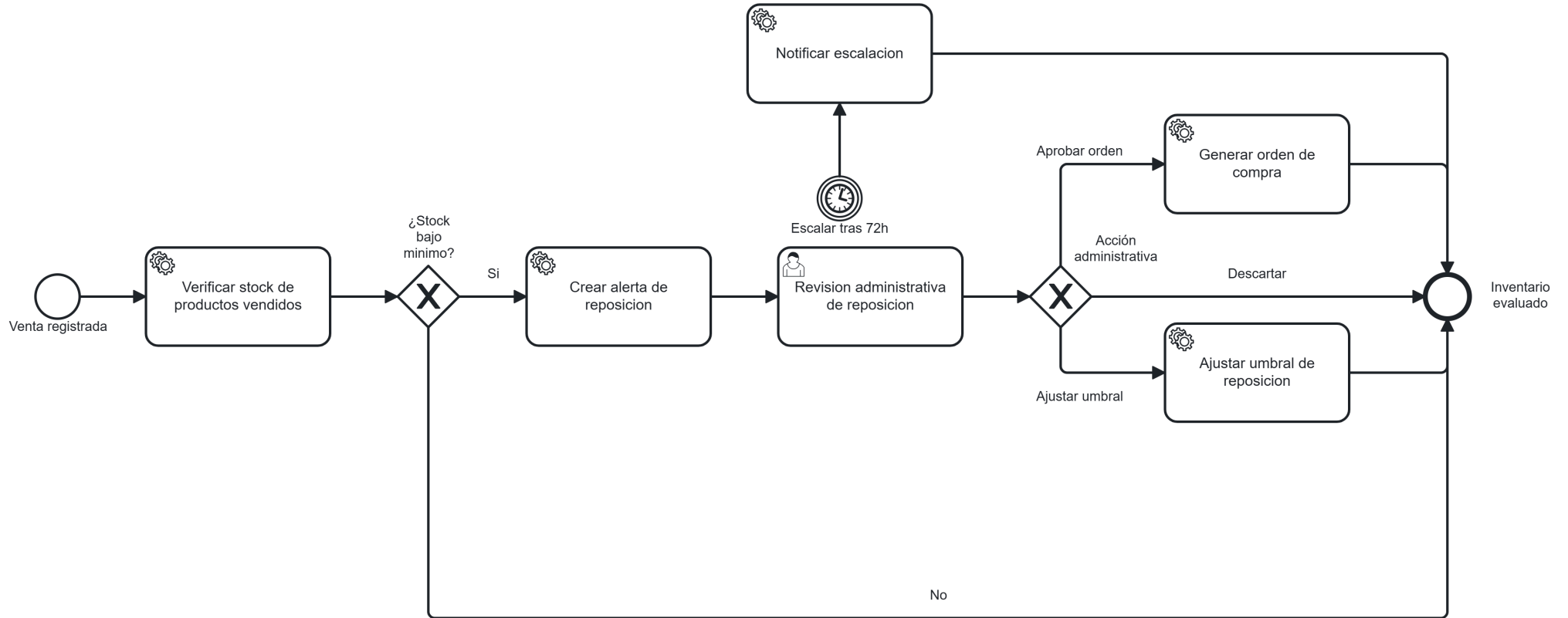


Figure 3: Alerta de inventario por venta

## 4.1 Procesos modelados con Camunda 8

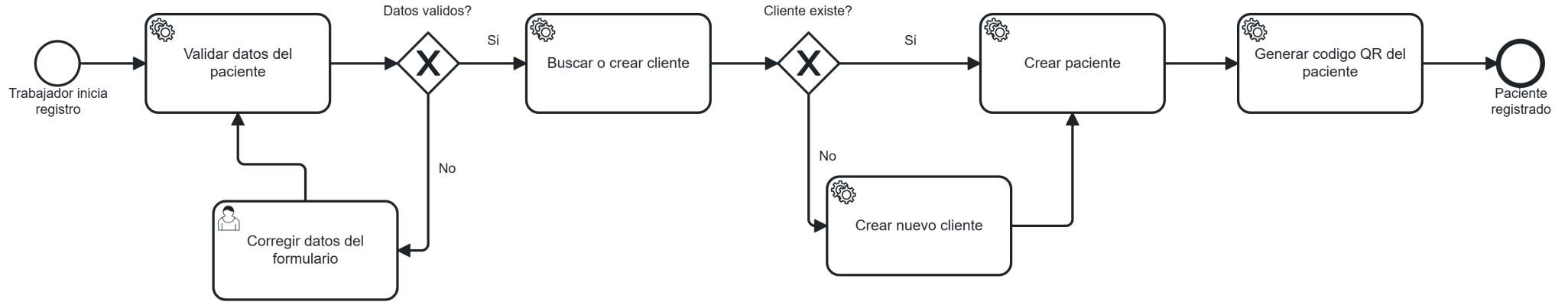


Figure 4: Registro de paciente

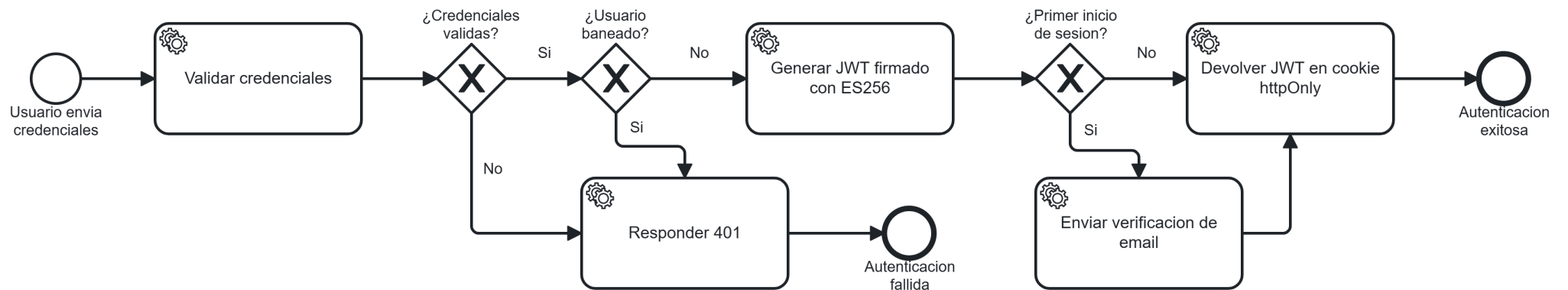


Figure 5: Flujo de autenticación y autorización

## 5. Seguridad Implementada

---

## 5.1 Autenticación con JWT (ES256)

### Flujo de autenticación:

- Credenciales → POST /api/auth/sign-in/email
- Better Auth valida contra la tabla user de PostgreSQL (hash bcrypt)
- Sesión en Redis + JWT firmado con ES256 (curva P-256), clave privada almacenada en la tabla jwks
- Claims: sub (UUID), role (admin/veterinarian/worker), iss, aud, exp, iat
- Token en cookie httpOnly y en header Authorization: Bearer {token}

### Validación autónoma:

```
mp.jwt.verify.publickey.algorithm=ES256  
mp.jwt.verify.publickey.location=http://localhost/api/auth/jwks  
smallrye.jwt.claims.groups=role
```

Cada servicio obtiene la clave pública del endpoint JWKS y verifica la firma sin depender del Auth Service. TracingFilter y AuditLogFilter leen sub y role para los registros de auditoría y el MDC.

## 5.2 Roles y autorización

| Rol                        | Permisos                                  | Módulos                                               |
|----------------------------|-------------------------------------------|-------------------------------------------------------|
| Administrador (admin)      | Acceso total: CRUD completo               | Usuarios, pacientes, clientes, especies, razas, citas |
| Veterinario (veterinarian) | Acceso clínico: leer, crear, actualizar   | Historial médico, citas                               |
| Trabajador (worker)        | Acceso operativo: leer, crear, actualizar | Clientes, pacientes, citas                            |

Autorización en dos capas: @RolesAllowed en controladores REST (DELETE solo admin) y TracingFilter (prioridad AUTHENTICATION + 100) para propagar who y role. Token inválido → 401; sin permisos → 403.

## 5.3 Propagación del token y seguridad en frontend

**Comunicación inter-servicios.** Mismo JWT del usuario propagado vía Authorization. correlationId generado por TracingFilter se propaga en X-Correlation-Id para correlacionar eventos en Loki y Grafana.

### **Seguridad en el frontend:**

- Token exclusivamente en cookie httpOnly con Secure y SameSite=None (anti-XSS y CSRF)
- Better Auth gestiona la renovación automática del token antes de que expire
- CORS restringido a tauri://localhost, http://tauri.localhost y http://localhost:1420
- Sesiones en Redis con prefijo sanipatitas:auth: para invalidación centralizada

## 6. Conclusiones

---

## 6.1 Conclusiones

- Tres servicios backend independientes (Auth, Patient, Appointment), cada uno con su stack y base de datos propios, compartiendo librerías comunes
- La autenticación con JWT + ES256 + JWKS permite validación descentralizada: cada servicio verifica tokens de forma autónoma sin depender del Auth Service en cada petición
- La auditoría transversal en tres capas (logs en tiempo real + Hibernate Envers + Loki/ Grafana) deja evidencia estructurada de toda operación crítica con trazabilidad extremo a extremo mediante `correlationId`
- El modelado BPMN con Camunda 8 documenta formalmente 5 procesos de negocio: auditoría, citas, inventario, registro de pacientes y autenticación
- El frontend en Astro 6 + React 19 consume los servicios mediante un cliente tipado generado automáticamente desde OpenAPI, eliminando duplicación manual y garantizando sincronización con los contratos del backend

## 6.1 Conclusiones

- La seguridad se aborda en múltiples capas: JWT ES256, bcrypt, roles con @RolesAllowed, CORS restrictivo, cookies httpOnly y sesiones centralizadas en Redis

**Gracias**